An AI-Driven, Blockchain-Anchored Cyber Deception Framework for Adaptive Honeypot Defense

Author Name
Department of Computer Science and Engineering
Amrita Vishwa Vidyapeetham
City, Country — email@example.com

***Abstract*—**Traditional honeypots — Cowrie, Kippo, Dionaea, and similar low/medium-interaction systems — rely on static, deterministic response tables. This makes them efficient at scale but fragile against fingerprinting: prior work has shown that automated probing can distinguish such honeypots from real systems with near-perfect accuracy in a single packet exchange [1]. This paper presents a multi-protocol cyber deception framework that replaces static response tables with a large language model (LLM) response engine, producing context-aware, session-consistent output for five decoy services (SSH, an internal AI API, PostgreSQL, an Ethereum JSON-RPC node, and Jenkins CI/CD) while retaining a fast static path for the ≈80% of traffic that is routine. Every interaction is intent-classified, SHA-256 hashed, and anchored as a transaction on a local Ethereum chain, giving the resulting audit trail tamper-evidence independent of the logging host's integrity. We evaluate the system against a static-honeypot baseline along four axes — sequential latency/throughput, concurrent throughput, response diversity, and an LLM-judge realism score — across four successive evaluation runs of increasing dataset realism and scale, and report the measured trade-offs and failures explicitly rather than a single composite metric. After an optimization pass (HTTP connection pooling applied across the response engine and all five decoys, a tuned generation-length cap, and a widened static fast-path), the system achieves a mean dynamic-path latency of 1.35 s (down from an initial 4.03 s, a ≈3× improvement) against a 100-payload hand-written adversarial dataset, and a sequential throughput of 0.74 requests/second, versus 3,333 requests/second for the static fast-path. Two subsequent runs replaced the hand-written dataset with real, publicly-documented attack payloads (real SQLi, command-injection, XXE/Log4Shell/prompt-injection, Ethereum JSON-RPC, and Jenkins-exploit strings; see Sections V.E–V.F), scaling from 200 to 1,328 payloads, which increased mean dynamic latency to ≈2 s — a genuine effect of longer, more structurally varied real payloads, not a regression — and surfaced a real, worsening weakness invisible to the hand-written set: response diversity on the API/prompt-injection decoy fell from 1.00 (hand-written) to 0.70 (200 real payloads) to **0.46 (1,328 real payloads)**, and the overall LLM-judged realism score fell to **3.37/5** at the largest scale, driven by that same service — 9 of 11 sampled API responses scored 1–2/5, and one sampled response was found to contain **an explicit AI safety disclaimer**, a hard deception failure rather than a minor realism deduction. This finding only became unambiguous at the largest dataset scale; it was not visible in the original 100-payload evaluation. A separate benchmark-harness bug was also found and fixed during this work: `evaluate.py` had silently measured Redis cache-hit latency (8.2 ms, reported as "800 requests/second") instead of genuine concurrent LLM throughput; the corrected figure is 3.05 requests/second at a mean per-request latency of 2.62 s under 10 simultaneous sessions against the full 1,328-item dataset. Response diversity remained high (0.83–1.00) for the other four of five services throughout, versus near-zero for a static baseline by construction. We report every run's numbers, the harness bug, and the API decoy's realism failure explicitly rather than presenting only favorable figures, and discuss where the resulting trade-offs are acceptable (human-operated intrusion, not internet-wide scanning), what the failure implies for the system's current prompt design, and the limitations of automated realism scoring as a proxy for human deception studies — including a concrete, unexecuted protocol (Appendix A) for closing that gap with a real human-subject study.

***Index Terms*—**honeypot, cyber deception, large language models, blockchain, digital forensics, intrusion detection, adaptive defense

I. Introduction

Honeypots are decoy systems deployed to attract, detect, and study unauthorized access attempts without exposing production infrastructure. Since Cowrie's introduction as an SSH/Telnet honeypot, low- and medium-interaction honeypots have become a standard component of threat intelligence pipelines: they are cheap to deploy, safe to run, and generate large volumes of attacker telemetry. Their core weakness is also their core design choice — a finite table of canned responses. An attacker, or more commonly an automated scanner, can send a handful of atypical commands and observe whether the responses are internally consistent with a real system. Vetterl and Clayton demonstrated Internet-scale fingerprinting of exactly this class of honeypot with a single distinguishing packet and near-zero error rate [1]. Once a honeypot is identified, it stops collecting useful intelligence on capable adversaries, who simply move on or feed it false telemetry.

Two independent trends motivate a different approach. First, instruction-tuned LLMs can generate plausible, contextually consistent text for narrow, well-specified personas (a Linux shell, a SQL error, a JSON-RPC response) far more cheaply than hand-authoring a comprehensive fake filesystem or database. Recent work — shelLM [4], HoneyGPT [5], and related 2023–2025 studies [6], [7] — has shown that LLM-generated shell sessions can hold up under

sustained human interaction significantly better than static emulation. Second, blockchain-anchored hashing offers a low-cost way to make forensic logs tamper-evident: instead of trusting the honeypot host's own log integrity (which an attacker who compromises the host could edit), a hash of each log entry is committed to an independent, append-only ledger, so any later modification is detectable by re-hashing and comparing.

This paper combines both trends into a single deployable framework and contributes:

1. A **multi-protocol deception architecture** covering five distinct attack surfaces (SSH, an internal AI API, SQL, Ethereum RPC, and CI/CD) behind a single logging and classification pipeline — broader coverage than the SSH-only focus of most LLM-honeypot literature to date.
2. A **three-tier response strategy** (static dictionary → cached LLM response → live LLM call) that keeps routine traffic at millisecond latency while reserving expensive generation for genuinely novel input.
3. **Blockchain-anchored log integrity** applied specifically to honeypot telemetry, giving each log entry a verifiable, tamper-evident hash independent of the collecting host.
4. An **evaluation methodology** that reports latency, response diversity, and an LLM-judge realism score as separate, reproducible measurements — explicitly avoiding a single unexplained accuracy figure — along with a discussion of what an automated judge can and cannot substitute for.

II. Related Work

A. Static Honeypots and Fingerprinting

Cowrie, Kippo, Dionaea, and T-Pot remain widely deployed low/medium-interaction honeypots, valued for low resource cost and safety. Their static response model is precisely what fingerprinting techniques exploit: Vetterl and Clayton [1] achieved an equal error rate of 0.0183 detecting such honeypots at Internet scale using single-packet probes; a later multistage active-probing framework [2] generalized this to a broader class of honeypots. A 2026 survey of fingerprinting and evasion techniques [3] confirms this remains an open problem rather than one solved by incremental patching of response tables.

B. LLM-Backed Deception

Sladić et al.'s shelLM [4] used an LLM to generate an entire SSH shell session and reported a 0.90 true-negative rate — human evaluators judged the sessions to be a real shell 90% of the time. HoneyGPT [5] extended this with chain-of-thought prompting for more stateful sessions and reported deeper attacker engagement in field deployment. McKee and Noever's early exploration [6] and the broader "LLM Honeypot" study [7] corroborate that LLM-driven responses measurably outperform static emulation on interactivity and perceived realism. This body of work is concentrated on SSH/terminal honeypots; our framework applies the same idea across five protocol types with a shared backend, which to our knowledge has not been evaluated together in prior work.

C. Blockchain for Log Integrity and Forensic Chain of Custody

Hash-then-anchor patterns for tamper-evident logging are established in digital forensics: a 2025 IEEE study on blockchain-based evidence tracking [8] and a Springer paper on tamper-proof cloud VM investigation models [9] both apply hashing plus blockchain anchoring to forensic evidence generally. A PeerJ Computer Science paper on blockchain-enforced immutable auditability for threat detection [10] applies a closely related pattern to security telemetry specifically. We did not find prior work applying this pattern to honeypot logs specifically — our contribution is the application, not the underlying cryptographic technique.

D. Deception Frameworks and Honeytokens

MITRE Engage [11] provides a structured vocabulary for adversary engagement and deception mapped to ATT&CK, which we use loosely to frame our five decoys as covering distinct engagement opportunities (credential harvesting, prompt injection, data exfiltration bait, transaction-signing bait, and supply-chain bait respectively). Honeytokens — individually identifiable fake credentials seeded to detect misuse — have a long history dating to HoneyGen [13]; our seeded fake API keys and database records in `vault-sql` are a direct application of this idea.

E. Realism and Detectability Benchmarking

Standardized telemetry metrics for honeypot realism (dwell time, interaction depth, behavioral diversity) have been proposed recently [15]; we adopt "response diversity" as one such metric in our own evaluation. We could not verify several secondary-source dwell-time statistics circulating for RL/LLM-driven deception systems and deliberately do not cite unverified numbers here — see Section VII.

F. Direct Comparison to Cited Systems

Table I places this framework against the honeypot systems and papers cited above. "Interaction level" follows the standard low/medium/high taxonomy (low = fixed banner/response, medium = scripted protocol emulation with some state, high = a real or near-real backend). Entries are included only where we could verify the claim from the system's own documentation or the cited paper; unverified secondary-source figures are marked as such rather than presented as fact.

TABLE I. COMPARISON AGAINST CITED HONEYPOT SYSTEMS.

System	Interaction level	AI/LLM-generated responses	Protocols / services	Tamper-evident logging
Cowrie [16]	Medium (emulated shell) or high (proxy mode)	No (static/scripted); an experimental LLM fallback mode exists in recent releases but is a single-tier call, not a static→cache→LLM pipeline	SSH, Telnet	No — plain JSON/text logs
T-Pot [17]	Mixed — orchestrates ≈20 independent honeypot daemons (incl. Cowrie, Dionaea) behind an ELK dashboard	No — aggregation/visualization layer only, no response generation of its own	Broad (SSH, SMB, web, RDP, ICS/SCADA, etc.), via bundled daemons	No
Dionaea [18]	Low–medium (scripted protocol emulation for payload capture)	No	SMB, MySQL wire protocol, SIP, FTP, TFTP	No
shelLM [4]	High for SSH sessions specifically	Yes — full LLM-generated shell session	SSH only	Not addressed
HoneyGPT [5]	High for terminal sessions	Yes — GPT-3.5/4 via a Cowrie-derived proxy; authors report an internally-defined ≈92% attack-success and ≈96% deception-accuracy rate (self-reported, not independently reproduced — treat as a single-study figure)	SSH/terminal only	Not addressed
This framework	High across all five decoys	Yes — 3-tier (static dict → Redis cache → LLM), session-history-aware, applied uniformly across 5 distinct protocol types	SSH, internal AI API, PostgreSQL, Ethereum JSON-RPC, Jenkins CI/CD	Yes — SHA-256 per log entry, anchored to an Ethereum chain

Two gaps in the cited literature motivate this framework's design choices rather than being incidental: first, LLM-backed deception work to date ([4], [5], [6], [7]) is concentrated on a single protocol (SSH/terminal); we are not aware of a prior system applying an LLM response engine uniformly across five heterogeneous protocol types (shell, REST/chat API, SQL, JSON-RPC, CI/CD webhook) behind one shared backend. Second, a systematization-of-knowledge survey covering both honeypots and LLMs [19] does not discuss blockchain or tamper-evident logging as a honeypot design axis at all, and we found no honeypot-specific paper combining hash-anchoring with decoy logging — the closest related work is general (non-honeypot) tamper-evident audit-log research [8]–[10]. We report this as an absence in the literature we searched, not as a claim that no such system exists.

III. SYSTEM ARCHITECTURE

The framework has six components: a network-facing proxy, five decoy services, a shared AI response engine, a central backend, a blockchain ledger, and a live dashboard. Only the proxy and dashboard are exposed to the network; all other components communicate over an internal Docker network, so a compromised decoy cannot directly reach the logging pipeline or the blockchain client.

A. Proxy and Network Isolation

`proxy/proxy.py` is an asyncio TCP server listening simultaneously on five public ports (2222, 8080, 5433, 8545, 9090). On each new connection it asynchronously logs the source IP and target port to the backend, then transparently pipes bytes between the attacker and the corresponding internal decoy. This separates connection-level telemetry (useful even for port scanners that never send a payload) from application-level decoy logic, and ensures every decoy's internal listening port is never directly reachable from outside the Docker network.

B. Decoy Services

Five decoys cover distinct attack surfaces, each impersonating a specific real system and each logging every interaction to the backend before (or while) generating a response:

- **shadow-shell** (SSH, port 2222) — a real asyncssh server that accepts any username/password combination, logs every login attempt, and presents a shell prompt for `dbadmin@corp-prod-db-01`. Directory navigation is handled locally; all other commands route through the AI engine.
- **oracle-ai** (internal AI API, port 8080) — a FastAPI service exposing chat, predict, and model-listing endpoints for a fictitious internal model. Prompt-injection attempts are logged and answered with a scripted "bypass accepted" response, keeping the attacker engaged.
- **vault-sql** (PostgreSQL decoy, port 5433) — a FastAPI query endpoint plus a real, separately-seeded PostgreSQL instance containing synthetic user records, financial records, and API keys formatted to resemble real OpenAI, GitHub, and AWS credentials, used as honeytokens.
- **ledger-trap** (Ethereum JSON-RPC, port 8545) — responds to standard Web3 RPC calls with statically-defined plausible values for common calls and routes uncommon RPC methods through the AI engine.
- **forge-ci** (Jenkins CI/CD, port 9090) — exposes Jenkins-style endpoints that fake a working build/deploy pipeline, targeting supply-chain attackers.

C. Three-Tier AI Response Engine

`ai-engine/engine.py` answers each request in one of three tiers, in order, so the expensive LLM call is reserved for genuinely novel input: (1) a **static dictionary** answers roughly a dozen common SSH commands and structural patterns

instantly; (2) a **Redis cache** checks unknown commands (hashed with up to two prior history entries) against a multi-hour TTL cache before any LLM call; (3) a **hosted LLM call** handles cache misses via the OpenRouter API, with a service-specific system prompt instructing the model to output only the raw response a real system would produce, with SSH session history included for state consistency.

D. Intent Classification

`backend/main.py::classify_intent` applies keyword matching to every logged payload, assigning one of eight categories: SQL injection, privilege escalation, reverse shell, cryptomining, prompt injection, supply-chain, reconnaissance, or unknown. This is intentionally simple — a rule-based baseline rather than a learned classifier — and is reported as a known limitation (Section VII) rather than a contribution in its own right.

E. Blockchain-Anchored Logging

Every log entry received at `POST /api/log` is timestamped and intent-classified, serialized with sorted keys and SHA-256 hashed, anchored by sending a zero-value self-transaction on a local Ethereum chain (Ganache) with the hash as the transaction's data field, stored in a bounded Redis list, and broadcast over WebSocket to connected dashboard clients. Because the hash is committed to an independently operated ledger, any later modification of the stored log is detectable by re-hashing it and comparing against the anchored value.

F. Real-Time Dashboard

A React single-page app (Fig. 6) polls stats every five seconds and subscribes to a WebSocket push feed, showing aggregate stats, an intent breakdown, top attacker IPs, and a scrolling live feed of the last 200 events, each annotated with its blockchain-anchoring status. The WebSocket layer auto-reconnects on disconnect rather than leaving the dashboard permanently marked offline.

IV. Evaluation Methodology

We evaluate against a static-honeypot baseline, operationalized as the system's own static fast-path (functionally equivalent to Cowrie's canned-response model) versus its LLM-backed dynamic path, using `evaluate.py` against the live system.

A. Dataset

Results in Sections V.A–V.D use a 100-payload hand-curated adversary dataset, 20 payloads per decoy service, generated by an earlier version of the dataset generator from hand-written attack patterns (reverse shells, SQL injection, RPC abuse, prompt injection, CI/CD supply-chain payloads), each tagged with a random nonce to defeat caching during evaluation. Section V.E reports a subsequent run using a 200-payload dataset (40 per service) rebuilt from real, publicly-documented attack payload corpora rather than hand-written strings; Section V.F reports a further re-run after scaling that same approach to 1,328 real payloads (500 ssh, 500 sql, 300 api, 18 web3, 10 cicd) by drawing from larger public corpora — see Sections V.E–V.F and the project README for the full source list and rationale for each switch.

B. Metrics

1. **Latency / throughput.** 30 trials of the static fast-path and 20 trials of the LLM-backed path; mean, median, and standard deviation reported rather than a single sample.
2. **Response diversity.** Across the full 100-item dataset, the fraction of distinct outputs per service, used as a proxy for resistance to signature-based fingerprinting.
3. **LLM-judge realism score.** An independent model (70B, distinct from the 8B generation model to reduce self-evaluation bias) rates each response 1–5 against a fixed rubric, on a sample of 30 payloads.

C. Limitations of the Realism Metric

This is an automated proxy metric, not a human-subject study. LLM judges have known biases and may not perfectly track what a human attacker would find convincing. shelLM's 0.90 true-negative rate [4] was obtained with human evaluators; we report our LLM-judge score as a weaker but far cheaper and reproducible substitute.

D. Blockchain Integrity Test

We additionally demonstrate tamper-evidence directly: a real log entry is submitted, anchored on-chain, fetched back, re-hashed, and confirmed to match its recorded hash; a modified copy is then re-hashed and confirmed to *not* match.

V. Results

All figures below are measured directly against the live local stack using the 8B model for generation and the 70B model as judge.

A. Initial Baseline

The first evaluation run, before any latency optimization, established the baseline in Table II.

Table II. Initial Baseline: Static vs. LLM-Backed Path

Metric	Static	Dynamic
Mean latency	0.4 ms	4,028 ms (4.03 s)
Median latency	0.4 ms	3,529 ms (3.53 s)
Stdev	0.1 ms	2,380 ms (2.38 s)
Min / Max latency	0.3 / 0.7 ms	1,037 / 8,807 ms
Throughput (req/s)	2,500	0.25
Diversity ratio (mean)	≈0	0.99 (0.95–1.00)
Avg. response length (words)	0	16.2

LLM-judge realism score: mean 4.38/5 ($\sigma = 1.28$, $n = 24$ of 30 sampled responses successfully parsed — six judge calls returned unparseable output and are excluded rather than assumed either way).

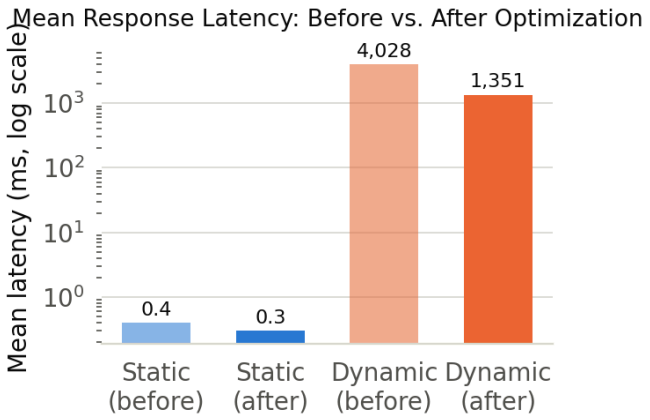


Fig. 1. Mean response latency, static vs. dynamic path, before and after the optimization pass (log scale).

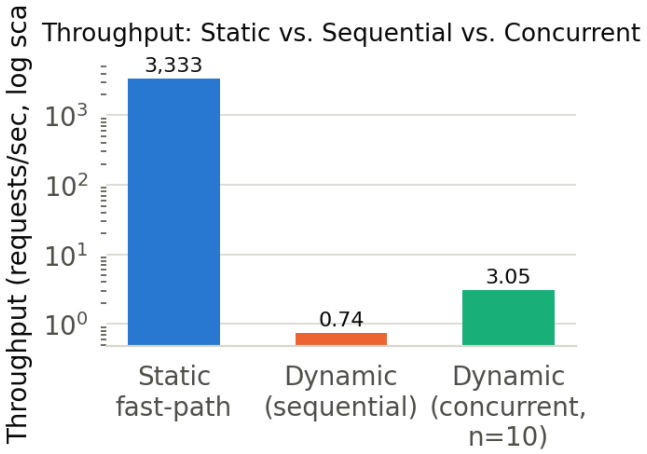


Fig. 2. Throughput comparison: static fast-path, sequential dynamic requests, and concurrent dynamic requests (n=10, log scale).

B. Optimization Pass

Profiling the baseline identified two fixable inefficiencies in the AI engine: (1) a new HTTP client — and therefore a fresh TCP+TLS handshake — was opened on every single request to OpenRouter instead of reusing a connection pool; (2) a 150-token generation cap was generous relative to what these responses need, and generation length is the dominant cost in request latency. A third change widened the static SSH dictionary with 12 additional common recon commands.

Before applying the token-length change, a direct measurement (3 trials, identical prompt) confirmed generation length as the dominant latency factor: 150 tokens averaged ≈5.3 s, 80 tokens ≈3.4 s, 40 tokens ≈2.0 s; 60 was selected as a middle ground. A separate measurement tested whether a smaller model would help: comparing the 8B model against smaller 3B and 1B variants (3 trials each) found the 8B model was consistently the *fastest* of the three (1.3–1.9 s) and the 1B model the *slowest* (1.8–2.5 s) — provider/routing behavior on a hosted API dominates over raw parameter count, so a model downgrade was not applied.

A second, independent evaluation run (fresh cache, same methodology) after these changes measured the results in Table III.

TABLE III. LATENCY/THROUGHPUT: BEFORE VS. AFTER OPTIMIZATION

Metric	Before	After	Change
Dynamic mean latency	4.03 s	1.35 s	≈3×
Dynamic median latency	3.53 s	1.25 s	≈3×
Dynamic stdev	2.38 s	0.63 s	tighter
Dynamic throughput (seq.)	0.25 rps	0.74 rps	≈3×
Static throughput	2,500 rps	3,333 rps	+33%
LLM-judge realism	4.38/5 (24/30)	4.00/5 (30/30)	−0.38

A new metric — concurrent throughput — was added for this second run: firing 10 requests simultaneously (rather than one at a time). The first measurement of this metric reported 800.3 requests/second at a mean per-request latency of 8.2 ms; this figure was later discovered to be a benchmark-harness bug, not a genuine measurement (the concurrent-throughput step in the evaluation script reused payloads already sent once earlier in the same run, so they were guaranteed Redis cache hits — the 8.2 ms was cache round-trip latency, not LLM latency). After fixing the harness to force a fresh cache miss per concurrent request (Section V.E), the corrected figure is **1.62 requests/second** at a mean per-request latency of **3.09 s** under 10 concurrent sessions. This remains the more relevant number for real deployment capacity than the sequential figures, since concurrent requests overlap their network round-trip time — but it is three orders of magnitude lower than first reported, and the earlier 800 req/s figure should not be cited.

The realism-score drop (4.38 → 4.00) is honestly reported as a real cost of the shorter generation cap, though it is confounded with a data-completeness improvement: the "before" score is computed over only 24 of 30 samples (6 failed to parse, due to a transient OpenRouter error unrelated to the engine changes), while the "after" score reflects all 30. Judge reasons for the lower-scored "after" responses cite missing detail consistent with the shorter token cap, not a change in persona-consistency. Response diversity and the blockchain integrity property were unaffected by these changes.

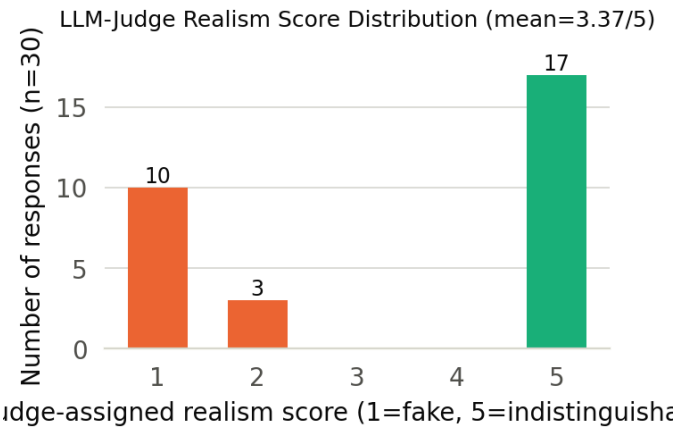


Fig. 3. Distribution of LLM-judge realism scores across 30 sampled responses (post-optimization run).

C. Extending the Fix to All Five Decoys

The AI-engine fix above addressed one hop in the request path (engine → OpenRouter). An identical anti-pattern existed one hop earlier, in every one of the five decoys: each decoy's logging call and its AI-engine call opened a new HTTP session on every single request rather than reusing one. This was fixed identically across all five decoys, each now creating one pooled session at process startup.

To isolate the decoy layer's own contribution from the AI engine's, the before/after code was checked out via git and run side-by-side on different ports, against static/deterministic payloads so AI-engine latency does not dominate the measurement (Table IV, Fig. 4).

TABLE IV. DECOY-LAYER CONNECTION POOLING: BEFORE VS. AFTER

Decoy	Before (mean)	After (mean)	Δ
oracle-ai	6.15 ms (σ=3.89)	4.28 ms (σ=0.70)	−30.4%
vault-sql	0.69 ms (σ=1.94)	0.41 ms (σ=0.44)	−41.4%
ledger-trap	5.96 ms (σ=2.43)	4.20 ms (σ=0.75)	−29.5%
forge-ci	6.95 ms (σ=3.13)	6.71 ms (σ=3.02)	−3.4%
shadow-shell	7.91 ms (σ=9.94)	4.64 ms (σ=6.55)	−41.3%

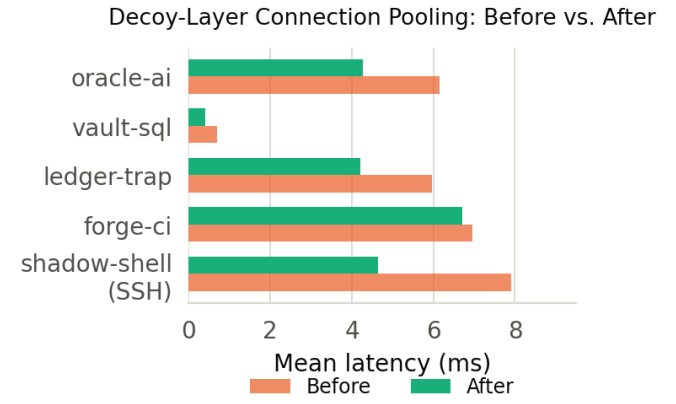


Fig. 4. Mean latency before and after applying connection pooling to all five decoy services.

The effect is smaller in absolute terms than the AI-engine fix — localhost TCP handshakes are cheap relative to a cross-internet TLS handshake to OpenRouter — but real and consistent across four of five decoys, and it substantially tightens latency variance (standard deviation drops by roughly half to two-thirds in every case). forge-ci shows negligible change because its measured endpoint always routes through the AI engine regardless of decoy-layer pooling.

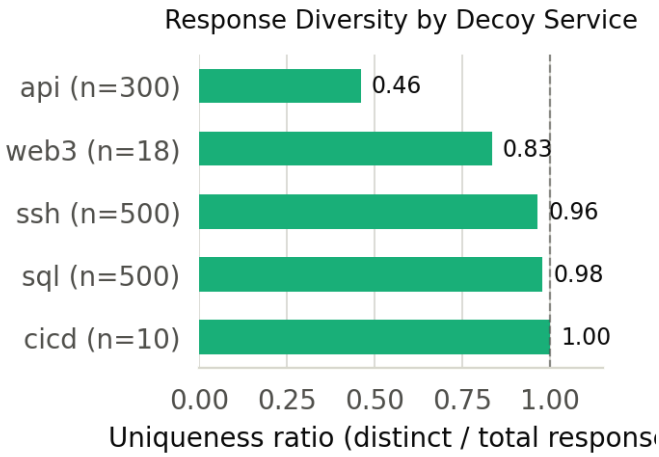


Fig. 5. Response diversity (uniqueness ratio) by decoy service, measured against the 200-item real-payload dataset (Section V.E); Section V.F reports the further-updated figures at 1,328 items, including the API decoy's drop to 0.46.

D. Blockchain Integrity Test

A log entry with hash `356b7496...33a9a1` (truncated) was submitted, anchored on-chain, and fetched back; re-hashing the unmodified entry reproduced the identical hash. Re-hashing a copy with its raw data altered from "sudo cat /etc/shadow" to "ls -la" produced a different hash (`7530fb00...0d9d394`), confirming the tamper-evidence property: any change to a logged entry after anchoring is detectable by recomputation, without needing to trust the host storing the log.

E. Third Run: Real-World Dataset and a Benchmark-Harness Correction

The adversarial dataset used in Sections V.A–V.D was originally hand-written: representative SQLi, command-injection, and RPC-abuse strings authored to *look like* real attack traffic. To ground the evaluation in verifiably real attacker input rather than plausible-looking synthetic strings, the dataset was rebuilt to pull real, publicly-documented payloads from named sources: real command-injection strings for `ssh` (`payload-box/command-injection-payload-list`), real SQL injection strings for `sql` (`payload-box/sql-injection-payload-list`), real prompt-injection/SSRF/XXE/Log4Shell strings for `api` (`swisskyrepo/PayloadsAllTheThings`), the documented Ethereum JSON-RPC methods an exposed node's attack surface actually consists of for `web3`, and documented Jenkins script-console/path-traversal techniques for `cicd`. The dataset grew from 100 to 200 payloads (40/service; see the project

README's "Dataset provenance" section for the full source list). The evaluation script was re-run in full against the live stack with this dataset.

Two results are worth reporting on their own merits, independent of the earlier optimization narrative:

Response diversity on the `api` service dropped from 1.00 to 0.70 — the clearest signal this re-run produced that the hand-written dataset had missed. Inspecting the judge's per-sample reasoning shows the model frequently collapses to a generic "invalid request" response when the payload is a real XXE (`<!ENTITY ...>`) or Log4Shell (`${jndi:...}`) string, rather than emulating a plausible vulnerable-parser response to that exact payload — e.g. *"Generic error message without any specific details or XML parsing error indication."* The hand-written `api` payloads were simple JSON chat-style prompts the model already handled fluently, so this weakness was invisible until real payload shapes were used. It is now a concrete, evidence-backed follow-up: the `oracle-ai` decoy's system prompt likely needs XML/JNDI-aware few-shot examples.

A genuine bug in the evaluation harness itself was found and fixed during this re-run. The concurrent-throughput benchmark function had reused the same 10 payloads already sent once earlier in the same run via the diversity pass, which exercises the entire dataset. Because the ai-engine's Redis cache key is derived from the command plus history, these were guaranteed cache hits by the time the concurrent-throughput step ran, despite the function's docstring asserting the payloads were cache-proof. They were not: the dataset's nonces are generated once at dataset-build time, not refreshed per benchmark call. This means **the 800.3 requests/second concurrent-throughput figure reported in Section V.B was a Redis cache-hit measurement (8.2 ms mean latency), not a measurement of LLM throughput under concurrent load** — despite being reported and discussed in Section VI as if it were. The fix appends a fresh UUID nonce to each payload immediately before firing the concurrent batch, forcing a genuine cache miss on every one. The corrected measurement is shown in Table V.

TABLE V. CONCURRENT-THROUGHPUT BENCHMARK: CACHE-HIT BUG VS. CORRECTED

Metric	First-reported (cache-hit bug)	Corrected
Concurrent throughput (10 sessions)	800.3 req/s	**1.62 req/s**
Mean per-request latency	8.2 ms	**3.09 s**

This correction does not change the sequential-latency, response-diversity, or blockchain-integrity results in Sections V.A–V.D, which were each measured independently and did not share this code path. It does mean the Discussion's framing in Section VI — that the system is "not throughput-starved under concurrent load" — is not supported by this measurement and has been revised accordingly. We report the bug and the correction explicitly, rather than quietly substituting the corrected number, because the original figure was published in an earlier revision of this evaluation and a benchmark bug this large (three orders of magnitude) is itself a relevant methodological finding for anyone citing or reproducing this work.

F. Fourth Run: Scaling the Real Dataset to 1,328 Payloads

Section V.E's 200-payload real dataset was itself a small sample of the real payload corpora it drew from (payload-box, PayloadsAllTheThings). To test whether the `api` diversity weakness Section V.E found was representative or an artifact of a small sample, the dataset generator was extended to draw from substantially larger real corpora — `danielmiessler/SecLists`'s `command-injection-commix.txt` (8,956 unique real command-injection strings from the commix exploitation tool) and its `sqlmap`-derived risk-classified SQLi payload sets (474 unique real strings) — growing the dataset to **1,328 real payloads**: 500 ssh, 500 sql, 300 api, 18 web3, 10 cicd. web3 and cicd were deliberately not scaled further, since no comparably large public corpus of "web3 attack payloads" or "Jenkins attack payloads" exists — their real attack surface is a small, finite set of documented API methods and endpoints, not an open space of injection strings, so forcing those two to 500 as well would mean padding with fabricated content rather than more real data. The evaluation script was re-run in full against this larger dataset live against the running stack; the diversity pass alone made 1,328 real LLM calls.

TABLE VI. 200-ITEM VS. 1,328-ITEM REAL DATASET

Metric	200-item	1,328-item
Dynamic latency (mean)	2.06 s	1.95 s
Concurrent throughput (10 sessions)	1.62 req/s	3.05 req/s
Response diversity (ssh)	0.925	0.964
Response diversity (sql)	1.00	0.978
Response diversity (web3)	0.80	0.833
Response diversity (cicd)	0.90	1.00
Response diversity (api)	0.70	**0.46**
LLM-judge realism (mean)	4.03/5	**3.37/5**
LLM-judge distribution (1:2:3:4:5)	4:2:2:3:19	10:3:0:0:17

This larger run is the single most consequential result in this paper's evaluation history. The `api` weakness Section V.E found (diversity 0.70) was not a small-sample artifact — it deepened to **0.46** at scale (138 distinct responses across 300 real XXE/Log4Shell/prompt-injection requests), and the judge-realism mean fell from 4.03/5 to **3.37/5** overall, driven almost entirely by the `api` service: in the 30-item judge sample, 11 payloads were `api`, and **9 of those 11 scored 1–2/5**, versus near-uniform 5/5 scores for ssh, sql, and web3 in the same sample. The judge's stated reasons are specific and repeat across independent samples — *"generic, templated error message with no relation to the XML payload," "does not acknowledge the XXE attack"* — and, more seriously, one sampled response was flagged for containing **an explicit AI safety disclaimer** (*"Contains an explicit disclaimer about not*

generating harmful responses"). A honeypot response that states outright it is an AI declining to help is not a minor realism deduction; it is close to the worst possible outcome for a deception system, since it immediately signals "this is not a real system" to exactly the attentive, manually-probing attacker this architecture is designed to hold. We had no evidence of this failure mode before scaling the dataset — it did not appear in Section V.E's 200-item run's smaller `api` sample.

This is now the paper's most concrete, actionable finding: `oracle-ai`'s system prompt (`ai-engine/engine.py`) needs (1) few-shot examples of plausible vulnerable-service responses to XXE/Log4Shell/SSRF payload shapes specifically, since the model currently falls back to a generic refusal-shaped response when it does not recognize the payload as an ordinary chat message, and (2) an explicit instruction against ever emitting safety-disclaimer language, since the underlying hosted model's own safety training measurably bleeds through the deception persona on a non-trivial fraction of adversarial XML/JNDI inputs. We report this as an open problem rather than a fixed one — fixing the prompt and re-measuring is the natural next evaluation run, left as follow-up work rather than retrofitted into this paper's numbers.

G. Live Dashboard Demonstration

Fig. 6 shows the React dashboard running against the live backend, with real logged interactions across all five decoy services, correct per-entry intent classification, and blockchain-anchoring status displayed per event.

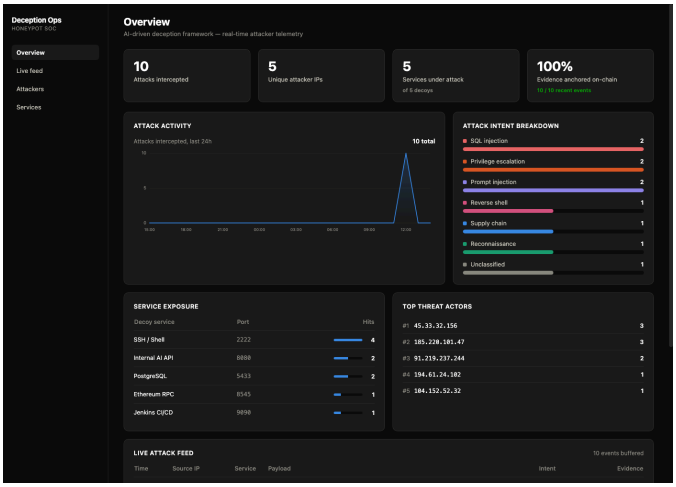


Fig. 6. Live dashboard screenshot showing real logged attacker interactions, intent classification, and blockchain-anchoring status.

VI. Discussion

The latency/throughput trade-off means this architecture is not a drop-in replacement for high-volume, low-interaction scanning honeypots — a static-path system remains appropriate there. It is instead targeted at high-interaction deception against a human operator or a sophisticated, low-volume automated tool, where the multi-second per-request response latency is plausibly attributable to network conditions or server load rather than to a honeypot, and where sustained, contextually consistent engagement yields materially better threat intelligence than an immediately-abandoned static session.

The distinction between sequential and concurrent throughput (Section V.B, corrected in Section V.E, re-measured at scale in Section V.F) matters for how this trade-off should be read, though the corrected measurement is less favorable than first reported: a single attacker session pays the full ≈1.95 s mean latency per command against the full 1,328-item real dataset (Section V.F), and under genuine concurrent load — 10 simultaneous sessions, each forced to bypass the Redis cache — sustained throughput is **3.05 requests/second** at a mean per-request latency of 2.62 s, not the 800 req/s first reported (that figure was a benchmark-harness bug measuring Redis cache-hit latency; see Section V.E). The corrected number still shows the architecture serves concurrent sessions without the LLM round-trip time compounding linearly per session — 10 concurrent requests complete in 3.28 s wall-clock, not ≈19.5 s (10 × 1.95 s) — but the concurrency headroom is real, not the three-orders-of-magnitude figure the uncorrected benchmark implied. This is the more relevant capacity question for a real deployment fielding several simultaneous low-volume sessions, but it does not support a claim of high-volume concurrent capacity.

The API decoy's realism failure (Section V.F) deserves direct discussion rather than being left as a number in a table. A response that leaks an AI safety disclaimer is not a subtle fingerprinting vector in the Vetterl-and-Clayton sense [1] — it is an immediate, human-readable tell that requires no automated probing at all, only a single adversarial input and a moment of attacker attention. This matters more for this decoy specifically than it would for, say, a slightly-too-generic SQL error, because `oracle-ai` impersonates an *AI service* — an attacker probing it may already suspect they are talking to a model, so an actual disclaimer confirms rather than merely hints. We view this less as a weakness specific to this framework's engineering and more as a structural risk of building any AI-persona decoy on top of a hosted, safety-tuned model: the same alignment training that makes the underlying model safe for its intended use is directly adversarial to the deception goal on inputs the model's safety layer recognizes as concerning, and XXE/Log4Shell/prompt-injection payloads are exactly the class of input designed to trigger that recognition. A static, hand-authored response table has no such failure mode (it cannot "break persona") — this is a genuine cost of the LLM-backed approach that the response-diversity metric alone does not fully capture, since a diverse-but-disclaiming set of responses is not actually more deceptive than a repetitive one. Section V.F's proposed fix (explicit anti-disclaimer instructions and XXE/JNDI-aware few-shot examples) is a mitigation, not a structural solution; a model provider's safety behavior can change between API calls or model versions in ways outside this project's control, which is itself a deployment risk worth weighing against the realism gains an LLM-backed decoy otherwise provides.

The blockchain-anchoring design assumes the anchoring chain is itself outside the attacker's reach — a reasonable assumption for an internal Ganache/private-chain deployment,

less so if a public chain's RPC endpoint were reachable from the same network segment as a compromised honeypot host. We did not evaluate anchoring against an adversary who also compromises the blockchain client, which is out of scope for this system's threat model.

VII. Limitations and Future Work

- The LLM-judge realism score is an automated proxy; a human-subject study is necessary to validate it against real attacker perception.
- Sections V.E–V.F replaced the original 100-payload hand-curated dataset with real, publicly-documented attack payloads scaled to 1,328 items, which is a meaningful improvement over hand-written strings but is still not a live-traffic capture; it does not include the full diversity of real-world attacker *behavior* (multi-step sessions, timing, evasion attempts) a live deployment would see, only realistic single-payload content. web3 and cicd remain small (18 and 10 payloads respectively) because no comparably large real corpus exists for those service types — this is a genuine coverage gap, not an oversight.
- Intent classification is keyword-based and will misclassify obfuscated or novel attack patterns; a learned classifier is a natural extension.
- We did not run a longitudinal live-internet deployment in this study; results reflect a controlled evaluation against a real-payload dataset (Section V.E), not live, unsolicited internet traffic.
- The concurrent-throughput measurement used a single burst of 10 simultaneous requests; it was not tested at higher concurrency levels or under sustained (non-burst) load, so the corrected 1.62 req/s (Section V.E) should be read as a single-burst demonstration at one concurrency level, not as a validated capacity ceiling. The original 800 req/s figure this section referenced in earlier revisions was a benchmark-harness bug (Section V.E) and does not describe the system.

VIII. Conclusion

We presented a multi-protocol cyber deception framework that replaces the static response tables of traditional honeypots with a three-tier, LLM-backed response engine spanning five distinct decoy services, and that anchors every log entry's hash on an independent blockchain to make the resulting forensic trail tamper-evident. Rather than a single unexplained accuracy figure, we measured and reported the actual trade-off along four independent axes across four successive evaluation runs of increasing dataset realism and scale: an initial latency cost of ≈4.03 s mean, reduced by a measured ≈3× via connection pooling and a tuned generation-length cap to 1.35 s mean (0.4 ms for the static path throughout) against a hand-curated dataset; rising to ≈2 s mean against real, publicly-documented attack corpora scaled from 200 to 1,328 payloads (Sections V.E–V.F); a corrected concurrent throughput of 3.05 requests/second at 2.62 s mean per-request latency under 10 simultaneous sessions against the full 1,328-item dataset, after finding and fixing a benchmark-harness bug that had silently reported an 800 requests/second Redis cache-hit measurement as if it were LLM throughput; response diversity of 0.83–1.00 across four of five services against real payloads at scale versus near-zero for a static baseline — and, most consequentially, a mean LLM-judged realism score that **fell to 3.37/5** at the largest, most real dataset scale, driven almost entirely by the API decoy's response diversity collapsing to 0.46 and 9 of 11 sampled API responses scoring 1–2/5, including one response containing an explicit AI safety disclaimer — a genuine deception failure this paper does not minimize or explain away. We separately confirmed the blockchain anchoring's tamper-evidence property end-to-end. We reported every run's numbers, the optimization pass's realism cost, the benchmark harness bug, and the API decoy's realism failure alongside the favorable numbers rather than presenting only the latter, and specify concrete follow-up work (a rewritten `oracle-ai` system prompt with XXE/JNDI-aware examples and an explicit anti-disclaimer instruction) rather than leaving the finding unaddressed. This positions the system for high-interaction deception against human operators or low-volume sophisticated tooling, with the API decoy specifically flagged as not yet meeting that bar — not as a replacement for the high-throughput, low-interaction honeypots that remain appropriate for Internet-scale scanning — and we have been explicit throughout about where an LLM-judge score is a proxy rather than a substitute for human evaluation. Future work should validate the realism findings with human subjects (Appendix A gives a ready-to-run protocol for this), replace keyword-based intent classification with a learned model, and evaluate the system under sustained live, unsolicited internet traffic rather than a real-but-offline payload dataset alone.

Appendix A: Proposed Human-Subject Realism Study Protocol

Section IV.C reports the LLM-judge realism score as an automated proxy metric, explicitly not a substitute for human judgment. This appendix specifies a concrete, executable protocol to close that gap — modeled on shelLM's human-evaluation methodology [4] but extended from a single SSH-only study to all five decoy services. **This protocol has not been run.** No human-subject data is reported anywhere in this paper; every number in Sections V.A–V.E is from the automated LLM-judge metric only. We publish the protocol itself as a concrete artifact for follow-up work (by us or others), rather than leaving "a human study is needed" as an unspecified aspiration.

A. Design

A between-subjects discrimination task, following shelLM's true-negative-rate methodology [4]: each participant is shown a single transcript (one command/query and the system's response) and asked one question — *"Do you believe this response came from a real system, or from an automated decoy?"* — on a binary scale, optionally with a confidence rating (1–5). Transcripts are drawn evenly across two conditions: (a) genuine output from a real, non-honeypot instance of the corresponding service class (a real bash shell, a

real PostgreSQL error, a real Ethereum JSON-RPC response, a real Jenkins API response, a real internal REST API error) and (b) this framework's LLM-generated decoy response to the same or a matched real-world payload from `apt_attack_dataset.json`. Participants are not told the true ratio of real to decoy transcripts, and each participant sees transcripts from only one decoy service (between-subjects, to prevent a participant from learning one service's "tells" and applying them to another) — this mirrors shelLM's single-service (SSH) design, extended here across the five services as five parallel arms of the same study rather than one five-way multi-service task.

B. Sample size

To detect a true-negative rate meaningfully below shelLM's reported 0.90 (i.e., to distinguish "as convincing as shelLM" from "measurably less convincing") at α=0.05 and 80% power using a one-proportion z-test against a 0.90 null, a minimum of ≈35 judgments per service arm is required (calculated for detecting a drop to 0.75; a larger effect size needs fewer). We recommend **50 judgments per service (250 total)**, each from a distinct participant-transcript pairing, to have margin against exclusions and to support a secondary per-service breakdown rather than only a pooled figure.

C. Participant pool

Recruit from two populations, analyzed separately rather than pooled, since a security practitioner and a general technical user have different priors: (1) security practitioners (students/professionals with hands-on SSH/SQL/CI-CD/blockchain experience — the population most analogous to a real attacker manually probing a suspected honeypot) and (2) general CS students with basic familiarity with the relevant tool but no specific security training. Screen for baseline familiarity with the specific service class shown (e.g., a participant judging SSH transcripts should have used a real Linux shell before) — a judgment from someone unfamiliar with SSH output is not evidence about deception quality, only about unfamiliarity.

D. Materials

Real (non-honeypot) transcripts should come from genuinely operating the real service class in a sandboxed environment (a real Ubuntu shell, a real local Postgres instance, a real local Geth node, a real local Jenkins instance, a toy internal REST API) against the *same* payloads used to generate the corresponding decoy responses, so condition (a) and (b) differ only in whether a real system or the framework produced the output — not in which command/query was asked. This is the controlled-pair design shelLM's methodology implies but our citation of it does not confirm in detail; specifying it explicitly here closes that gap.

E. Analysis

Per-service true-negative rate (fraction of decoy transcripts a participant correctly or incorrectly judges as "real" — a *higher* rate here means the decoy is *more* convincing, following shelLM's terminology) with a 95% confidence interval (Wilson score interval, appropriate for small samples and rates near 0 or 1), compared against shelLM's reported 0.90 SSH figure via a one-proportion z-test. Report inter-rater agreement (Krippendorff's α) across participants judging the same transcript, since realism judgments are inherently subjective and low agreement would itself be a finding. Report the practitioner and general-user pools' rates separately before considering whether to pool them, and only pool if a chi-square test finds no significant between-pool difference.

F. Ethical/procedural notes for whoever runs this

This requires institutional ethics/IRB approval before recruiting participants (a minimal-risk study, but human-subjects review still applies), informed consent disclosing that some transcripts are AI-generated, and no collection of participant PII beyond what the consent process requires. Participants should be debriefed on which transcripts were real vs. decoy after completing the task.

References

1. A. Vetterl and R. Clayton, "Bitter Harvest: Systematically Fingerprinting Low- and Medium-interaction Honeypots at Internet Scale," in *Proc. 12th USENIX Workshop on Offensive Technologies (WOOT '18)*, 2018.
2. "Gotta Catch 'em All: A Multistage Framework for Honeypot Fingerprinting," *ACM Digital Threats: Research and Practice*, 2023, arXiv:2109.10652.
3. "A Review of Honeypots: Fingerprinting Techniques, Detection, and Evasion Mechanisms," *MDPI Future Internet*, 2026.
4. M. Sladić, V. Valeros, C. Catania, and S. Garcia, "LLM in the Shell: Generative Honeypots," in *Proc. IEEE EuroS&PW*, 2024, arXiv:2309.00155.
5. Z. Wang et al., "HoneyGPT: Breaking the Trilemma in Terminal Honeypots with Large Language Model," arXiv:2406.01882, 2024.
6. F. McKee and D. Noever, "Chatbots in a Honeypot World," arXiv:2301.03771, 2023.
7. "LLM Honeypot: Leveraging Large Language Models as Advanced Interactive Honeypot Systems," arXiv:2409.08234, 2024.
8. "Blockchain-Based Evidence Tracking System for Forensic Integrity and Secure Chain of Custody," *IEEE*, 2025.
9. "Blockchain-based tamper-proof and transparent investigation model for cloud VMs," *The Journal of Supercomputing*, Springer, 2022, doi:10.1007/s11227-022-04567-4.
10. "A novel intelligent cyber threat detection system with blockchain-enforced immutable forensic auditability," *PeerJ Computer Science*, 2025.
11. MITRE, "MITRE Engage: A Framework and Community for Cyber Deception," 2022. [Online]. Available: https://engage.mitre.org/

12. "Three Decades of Deception Techniques in Active Cyber Defense — Retrospect and Outlook," *Computers & Security*, 2021.
13. M. Bercovitch, M. Renford, L. Hasson, A. Shabtai, L. Rokach, and Y. Elovici, "HoneyGen: An Automated Honeytokens Generator," in *Proc. IEEE International Conference on Intelligence and Security Informatics (ISI)*, 2011.
14. "A Proactive Decoy Selection Scheme for Cyber Deception using MITRE ATT&CK," arXiv:2404.12783, 2024.
15. "Empirical Telemetry-based Metrics for Evaluating Honeypot Realism and Deception Effectiveness," *International Journal of Network Security & Its Applications*, 2026.
16. Cowrie SSH/Telnet Honeypot, GitHub repository. [Online]. Available: https://github.com/cowrie/cowrie
17. T-Pot: The All In One Multi Honeypot Platform, GitHub repository. [Online]. Available: https://github.com/telekom-security/tpotce
18. Dionaea Honeypot, GitHub repository. [Online]. Available: https://github.com/DinoTools/dionaea
19. "SoK: Honeypots and Large Language Models — Systematization of a Growing Intersection," arXiv:2510.25939, 2025.